Winter 2026

STAT 390 - Presentation 1

Team 3

Leah Tan, Xiaolin Liu, Elsa Steen Koppell, Leslie Gonzalez-Flores, and Sofie Langenhuizen

Roles & Responsibilities

Our Team

Leah
Consultant

Responsible for suggesting changes and room for improvement in methodology, execution

Sofie
Coder

Expert in the code, produces and executes the model, making necessary edits to any redundancies

Xiaolin
Coder

Expert in the code, produces and executes the model, making necessary edits to any redundancies

Leslie
Mathematician

Expert in the math behind the model, suggest edits to methodology and provide justification for parameters

Elsa
Mathematician

Expert in the math behind the model, suggest edits to methodology and provide justification for parameters

Leah

Agenda

- Background & Context
- File Structure
- Environment Configuration
- Data Preprocessing
- Model Configuration
- Training & Evaluation

	Leah

Agenda

Analysis

Takeaways

Background & Context

We want to **develop a machine learning algorithm** that can **accurately classify** the categories of **C-MIL** (Conjunctival melanocytic intraepithelial lesions)

- ➔ Can manifest as 3 different classes: **benign** (no treatment necessary), **low-grade** (carefully observed), **high-grade** (aggressive treatment)

Why use ML?

- ➔ Very susceptible to human error, with detrimental costs
- ➔ Successful application in breast cancer classification
  - ◆ Dual-stream CNN-transformer model that jointly learns from different medical image stains
    - Inspired our own methodology
  - ◆ *Cheng, Y., Lama, N., Chen, M. et al. Synergistic H&E and IHC image analysis by AI predicts cancer biomarkers and survival outcomes in colorectal and breast cancer. Commun Med 5, 328 (2025)*

Code File Structure

Environment Configuration
requirements.txt, data_splits/, sbatch_files/, config.py, utils.py

Data Preprocessing
dataset.py, data_utils.py

Model Configuration
models.py

Training & Evaluation
trainer.py

Analysis
attention_analysis.py

Executed by main.py

Environment Configuration

- **requirements.txt**
  Contains dependencies required to execute code
- **data_splits/**
  Stores existing train/test/val splits, allowing for reproducibility and faster setup
- **sbatch_files/**
  Contains pre-configured scripts for submitting SLURM jobs to Quest to execute the script
- **config.py**
  Stores data paths and configuration parameters for models, training, and image manipulation
- **utils.py**
  Provides general purpose utility functions for MIL training and evaluation (e.g. seed/device setup, save/load data splits, etc.)

Data Preprocessing

➔ **data_utils.py**

Defines helper functions to perform data preprocessing:

- Groups patch files into slice and stain level bags for each case
- Outcome variable is made binary
- Cases are split into train, test, validation groups using stratification

Redundancy:

The **build_slice_to_class_map()** function maps every combination of (case_id, slice_id) to a class.

- This is redundant because every slice in the same case will have the same class.
- The build_slice_to_class_map() function should be rewritten as a build_case_to_class_map() function instead.
- This will also eliminate the need for creating a case-class label map in **split_by_case_stratified()** and **build_case_dict()**.

Data Preprocessing - Splits

➔ **data_utils.py**

Data splitting (along case_id) with **split_by_case_stratified()** function

- First split: train vs. temp (val & test)
- Second split: val vs. test (from temp)

Ratios established in **config.py** determine final allocation of cases to the three sets: training (60%), validation (20%), testing (20%)

Binary class labels made in **build_slice_to_class_map()**, are basis for stratification

- If raw label is 1.0 → 0 (benign)
- If raw label is 3.0 or 4.0 → 1 (high-grade)

report_no_leak() function confirms training, validation, testing sets all disjoint.

Data Preprocessing

➔ **dataset.py**

Defines **StainBagCaseDataset** class that serves as the input to our MIL model:

- The dataset takes in a **label_map** mapping each case ID to a class label and a **case_dict** mapping each case ID to stains and corresponding patches
- Returns each case with a class label and a dictionary of stains, where each stain maps to a list of slice tensors, and each slice tensor has shape (P, C, H, W) representing its corresponding patches

Also sets up data augmentation

Data Preprocessing - Math

Train Data Transformations

- **RandomResizedCrop**(resized to 224×224)
- RandomHorizontalFlip
- RandomVerticalFlip
- RandomRotation(+/ 15°)
- **RandomColorJitter**(0.2 brightness, contrast, hue)
  Although low-value used for contrast and hue, can lead to distortion of necessary cell information (nuclears, cytoplasm, cell body)
- **Normalized**

Val/Test Data Transformation

- **Resize (224×224)**
- **Normalized**

$$densenet121\ normalized\ pixel = \frac{pixel - mean}{std}$$

Model Configuration

➔ **model.py**

Defines the hierarchical attention-based MIL model

Creates two custom PyTorch modules:

- **AttentionPool**:

  Uses a small NN (**self.attention**) with layers Linear→Tanh→Dropout→Linear inside **.forward()** method to compute attention weights

  Applies softmax normalization to attention weights

  Applies weights to input embeddings and returns weighted embeddings

  Optionally returns attention weights

Model Configuration

➔ **model.py**

- **HierarchicalAttnMIL**:
  A 3-level hierarchical MIL model with attention at patch level, stain level, and case level using two main methods
  Sequentially processes slices (**.process_single_stain()**) and then stains (**.forward()**).
  Uses a pre-trained DenseNet-121 base model with a frozen feature extractor to get raw features
  Applies Adaptive Average Pooling to base model features
  Uses a patch projector (Linear→ReLU→Dropout) to get patch embeddings
  Calls **AttentionPool** at the patch level to get slice embeddings, then the slice level to get stain embeddings, then at the stain level to get case embeddings
  Perform a final classification on case embeddings with a linear classifier regularized with dropout
  Memory efficient throughout by periodically deleting objects that become obsolete

  Small fix:
  DenseNet pretrained parameter is deprecated
  - This class calls **densenet121(pretrained=True)** multiple times and returns warnings, should be replaced with the weights parameter

Model Configuration

➔ **model.py**

Defines one function:

- **create_model()**

  Loads configuration settings from config.py

  Returns a **HierarchicalAttnMIL** object based on configuration settings

  Gets called in main.py to create the MIL model

Model Configuration - Math Tensor Changes

*red is example of tensor change when image size (224×224)

Initial Patch Image Input
- (C,H,W) - 3 Channels (R,G,B), Height, Width
  (3,224,224)

Batch input into MIL Model (DenseNet121 Default)
- (B,M,C,H,W) - Batch, Patches, Channels, Height, Width
  (1,M,3,224,224)

After Convolution Layer
- (1,M,64,H/2,W/2)
  (1,M,64,112,112)

After Norm, Relu, and Pool Layer
- (1,M,64,H/4,W/4)
  (1,M,64,56,56)

After Denseblock 1 (6 Dense Layers)
- (1,M,256,H/4,W/4)
  (1,M,256,56,56)

After Transition 1 (Norm, Relu,Conv,AvgPool(2×2))
- (1,M,128,H/8,W/8)
  (1,M,128,28,28)

Model Configuration - Math Tensor Changes

After Denseblock 2(12 Dense Layers)
- (1,M,512,H/8,W/8)
  (1,M,512,28,28)

After Transition 2 (Norm, Relu, Conv, AvgPool)
- (1,M,256,H/16,W/16)
  (1,M,256,14,14)

After Denseblock 3 (24 Layers)
- (1,M,1024,H/16,W/16)
  (1,M,1024,14,14)

After Transition 3 (Norm, Relu, Conv, AvgPool)
- (1,M,512,H/32,W/32)
  (1,M,512,7,7)

After Denseblock 4 (16 Layers)
- (1,M,1024,H/32,W/32)
  (1,M,1024,7,7)

After Norm5 + relu
- (1,M,1024,H/32,W/32)
  (1,M,1024,7,7)

Model Configuration - Math Tensor Changes

After adaptive_avg_pooling2d(collapses to 2×2)

- (1,M,1024,2,2)

After flattening

- (1,M,4096)

Patch Projection

- (M,512) - linearly compressed to vector of 512 from default embedding n_dim. Converts features into patch emebddings

Attention Pooling

$$weighted\ avg.\ all\ embeddings:\ z = \sum_{k=1}^{k} a_k h_k$$

$$a_k = \frac{exp(W^T tanh(Vh_k^T))}{\sum_{j=1}^{k} exp(W^T tanh(Vh_j))}$$

Model Configuration - Math Tensor Changes

For each patch in slice:

$$u_k = Vh_k$$

$$= \sum_{j=1}^{512} V[i,j] \cdot h_k[j],\ i = 1,\ldots,128$$

$$h_k = [512]$$

$$V = matrix[128, 512]$$

a) Linear transformation onto embeddings
Produces new 1D Vector [128]

Model Configuration - Math Tensor Changes

$$v_k = tanh(u_k) = tanh(Vh_k)$$

b) Add non-linearity to vector. **[128]**

$$s_k = W^T tanh(Vh_k)$$

$$s_k = \sum_{i=1}^{128} W_i \cdot V_k[i]$$

c) Weight vector dot product with projected hidden layer after non-linearity added.
Each component of s_k is multiplied by learnable weight parameter in W. **[128]**

Model Configuration - Math Tensor Changes

$$a_k = \frac{exp(s_k)}{\sum_{j=1}^{m} exp(s_j)}, \; k = 1..., M$$

d) Convert scalar score of M patches in slice to a weight

From code, softmax is used to ensure weights add up to 1: $0<a_k<1$. The size of a_k is the size of one attention weight per patch in slice

$$\sum_{k=1}^{M} a_k h_k$$

e) Weighted Sum of Patches (1,512)

This provides slice embedding, composed of each patch score

Model Configuration - Math Tensor Changes

Attention scoring process is **repeated** for each **slice in stain - (1,512)**
- Gives **stain embedding**, composed of each slice score

Attention scoring process is **repeated** for every **stain per case - (1,512)**
- Gives **case embedding**, composed of each stain score

Higher attention scores contribute more to being in the next level

For classification
- **(1,512)** is linearly compressed to **(1,2)**
  Provides logits for each case
- **(1,2)** passed through a sigmoid function to provide predicted probability belonging to HG CMIL

Final output: (1,2) shape of predicted probabilities

Training & Evaluation

➔ **trainer.py**

Implements training/validation/evaluation utilities for the MIL model. This file wraps the model, optimizer, scheduler, loss function, and checkpointing into a single MILTrainer object. It also provides a helper function *count_patches_by_class* for data analysis.

Training & Evaluation

➔ Loss function

Fall 2025 final model uses (averaged) **cross entropy** loss with two classes (*C = 2*). Below is mathematical formula for calculation.

$$\text{CE Loss} = -\sum_{c=0}^{C} w_c y_c \log(\hat{y}_c)$$

$$C = \text{Number of classes}$$

$$w_i = \text{Weight for class c}$$

$$y_i = \text{One-hot encoding (1 if class c, 0 if not)}$$

$$\hat{y}_i = \text{Predicted probability for class c}$$

For each *case* within each class, the loss is calculated (see more details on next slide). Then, take average loss for each *class*.

Training & Evaluation

➔ **Loss function cont.**

Defined in constructor of **MILTrainer** class, **nn.CrossEntropyLoss()** function (from PyTorch) combines two existing functions for neural network ML, **nn.LogSoftmax()** and **nn.NLLLoss()**. Takes the raw logit input and returns loss, which is stored in the **criterion** attribute.

Only argument in the loss function that is currently specified is **weights**. The weight for the benign class has been increased from **2.0** to **2.5**. Weight for high-grade class is **1.0**.

Average (cross entropy) loss is used to evaluate models via the **evaluate()** function in **trainer.py**.

Potential modification: Adjusting the weights for benign/high-grade. Can increase the penalty for misclassifying rare classes, thus encouraging the model not to ignore them in pursuit of high accuracy.

Training & Evaluation

➔ **trainer.py**

MILTrainer Class

Attributes:

- **Model**
  - The model instance is initiated outside of this class
- Device: CUDA or CPU
- Checkpoint directory
- **Optimizer**
  - Using the *Adaptive Moment Estimation* algorithm to train, known for combining the benefits of momentum and adaptive learning rates; good for large datasets
- **Criterion**
  - Loss function for the trainer
  - Using *CrossEntropyLoss*
  - Initialized using class weights to handle imbalance
- **Scheduler**: adjusts the learning rate during training to improve model performance
  - Checks to see if training config tells us to use a scheduler; skip if not specified
  - Defaults to the *Reduce LR on Plateau* scheduler
  - Can also specify to be *Cosine* scheduler
- **[Early stopping attributes]**
  - Early stopping patience
  - Early stopping min delta
  - Best val loss
  - Epochs without improvement
- **[Training history attributes]**
  - Train losses
  - Val losses
  - Val accuracies
  - Learning rates

Training & Evaluation

➔ **trainer.py**

MILTrainer Class

Methods:

- **train_epoch()**
  - Trains the model for one epoch
  - Runs model on stain slices, computes loss, runs backward, and steps optimizer
  - Returns *average training loss*
- **validate()**
  - Valides the model and disables grad (gradient computation)
  - Similar input handling as *train_epoch*
  - Computes and returns average loss and accuracy (per-case)
- **save_checkpoint()**
  - Creates checkpoint directory if needed
  - Saves model state, optimizer state, (optional) scheduler state,
  - training history lists, best_val_loss, epoch, etc.
- **load_checkpoint()**
  - Loads checkpoint and restores certain model attributes to the states recorded in the checkpoint
  - Returns the saved epoch number
- **train()**
  - Orchestrate the full training loop
    - Reads epochs and early stopping settings
    - Calls *train_epoch, validate*, records learning rate, and prints metrics for each epoch
    - Saves checkpoints
    - Implements early stopping (*epochs_without_improvement* vs *early_stopping_patience*)
  - Prints progress and completion

Training & Evaluation

➔ **trainer.py**

MILTrainer Class

Methods cont.

- **evaluate()**
  - Evaluates the model on test set
  - For each example: run model, compute loss, softmax probabilities, and predicted label
  - Returns metrics dict: test_loss, test_accuracy, predictions, probs, etc.
  - Can also save a CSV of predictions and a confusion matrix PNG
- **_save_predictions_csv()**
  - Saves a *predictions.csv* with columns: case_id, true_label, predicted_label, prob_benign, prob_high_grade, correct.
- **_save_confusion_matrix()**
  - Creates and saves a confusion matrix image (confusion_matrix.png) using labels [Benign, High-grade]

**Not a class method*

- **count_patches_by_class()**
  - Utility for dataset analysis.
  - Iterates cases and stains, counts total patches per case, aggregates by class
  - Prints patch counts per class for the given split and returns the counts.

Pre-trained Parameters, Estimated Parameters, Tunable Hyperparameters

Pre-trained Parameters

- **DenseNet121 CNN filters** allow for feature extraction.
- **DenseNet121 Weights** (pre-trained on ImageNet Data)
  Given DenseNet121 is base model with frozen feature extractor, weights are not updated during training with calculated gradients.

Estimated (Learnable) Parameters

- **Patch Projection** converts CNN features into patch embeddings
- **Attention Pooling and Attention Scores** are calculated for each patch, slice, and stain of a case that impact low level embedding class or a case. Hierarchical structure of attention scoring means each score influences importance of score within next level
- **Classifier** is estimated through the final bag-level embedding and outputs from logits. Model has to convert bag-level embeddings into predictions

Pre-trained Parameters, Estimated Parameters, Tunable Hyperparameters

Tunable Hyperparameters

- **Learning Rate** determines how quickly model updates its parameters during training. Value is fixed, influenced by **Learning Rate Scheduler**
- **Weight Decay** (regularization technique)
  Prevents overfitting through large weight penalization
- **Class weights** (class imbalance)
  Forces model to assign different levels of importance to each class during training.Benign class receives larger penalty when misclassified
- **Epochs** represent the number of complete passes through the entire training dataset. Value is fixed, but number of epochs during training impacted by early stopping.
- **Learning Rate Scheduler** reduces learning rate value when validation loss stops improving in the model. Influenced by **number of epochs** models waits to consecutively see without improvement, **minimum learning rate value**, and **scheduler factor** that multiples learning rate value when validation loss has not improved
- **Early Stopping** stops model training when performance on validation data has not improved. Influenced by **patience value** the number of consecutive epochs model runs without improvement, **minimum number of epochs model must run**, and **minimum change** in monitored quantity to qualify as an improvement

To find best hyperparameter values, can implement grid search. However, this can be computationally expensive, especially with the large dataset being utilized.

Analysis

attention_analysis.py

Creates **summary statistics and visualizations** to help developers understand attention weights created by their models.

- **analyze_attention_weights():** Runs the model over the test set to collect attention weights and performs per-case analysis, visualization, and summaries
- **analyze_case_attention():** Extracts case and stain level attention, identifies most/least attended slices, visualizes slice patch for a single case
- **visualize_patch_attention():** Creates and saves a grid image showing the top/bottom N patches for one slice annotated with their attention weights
- **visualize_case_effective_patches():** Selects top/bottom patches by effective attention across a case and saves montage images
- **save_attention_summary():** Writes a human readable text file summarizing per-case attention highlights
- **compute_effective_patch_attention():** Loads and formats attention weights and calculates combined weight effect per patch
- **plot_effective_patch_attention_distribution_per_case():** Plots distribution of patch-level combined weight effects for each case
- **analyze_top_effective_patches_per_case():** Summarizes most important patches, slices, and stains
- **plot_slice_attention_distribution_per_caseandstain():** Plots distribution of slice-level attention weights for each case/stain

Possible Redundancy:
visualize_patch_attention() and **visualize_case_effective_patches()** essentially show the same thing but at slice versus case level
It seems unnecessary to see the top and bottom patches at the slice level
Propose only returning at the case level

Takeaways & Next Steps

➔ Imbalanced Attention Weights

- Among 3 stain types, **H&E dominates** the vast majority for each slice

	H&E	Melan	Sox10
Test	409.2	250.9	238.1
Train	470.5	263.2	219.9
Val	504.8	241.9	172.8

```
Case 84:
----------------------------
Most attended stain: melan

Stain-level attention:
  h&e: 0.4547
  melan: 0.5007
  sox10: 0.0446
```

- This translates to the attention weights assigned to case classification, potential **room for overfitting to patterns captured in one stain**
  - Among top 5% of most learnable cases, exclusively dominated by H&E or Melan
    - "For difficult cases, particularly desmoplastic melanoma, SOX-10 is the preferred marker"
      - *Dass SE, et al. Comparison of SOX-10, HMB-45, and Melan-A in Benign Melanocytic Lesions. Clin Cosmet Investig Dermatol. 2021 Oct 5;14:1419-1425.*

Takeaways & Next Steps

- **Tuned Hyperparameters**
  - Currently arbitrarily chosen, despite ability to significantly **alter how model trains** itself & **penalizes class imbalance/overfitting**
    - **Gridsearch**, although computationally expensive, allows room to tune to best hyperparameters
    - **Bayesian Hyperparameter Tuning** strikes best balance of performance & computation time
      - Build probabilistic model of objective function (e.g. loss, accuracy), iteratively repeat process based on previous results

Takeaways & Next Steps

- **Loss Function Adjustment**
  - Currently uses **Cross Entropy,** with adjusted weighting to **penalize benign cases more than high-grade** (2.5: 1.0)
  - Room to adjust weights further to avoid misclassifying/overlooking classes in pursuit of accuracy
  - **Focal Loss**
    - Modification of CE that **down-weights easy-to-classify examples** (high-grade) and pays **more attention to difficult cases** (low-grade)
    - Specifically utilized for highly imbalanced datasets

$$FL = -(1 - P_t)^{\gamma} \log(P_t)$$

$$CE = -\log(P_t)$$

Takeaways & Next Steps

- Coding Efficiency
  - <u>Redundancy:</u>
    - **build_slice_to_class_map()** unnecessarily maps every combination of (case_id, slice_id) to a class
      - Redundant since every slice will have the same class, can rewrite as case to class
    - **visualize_patch_attention()** and **visualize_case_effective_patches()** show same thing, only need case level
  - <u>Deprecation:</u> **Densenet121(pretrained=True)** no longer used function, generates too many errors potential issues

Thank you!